

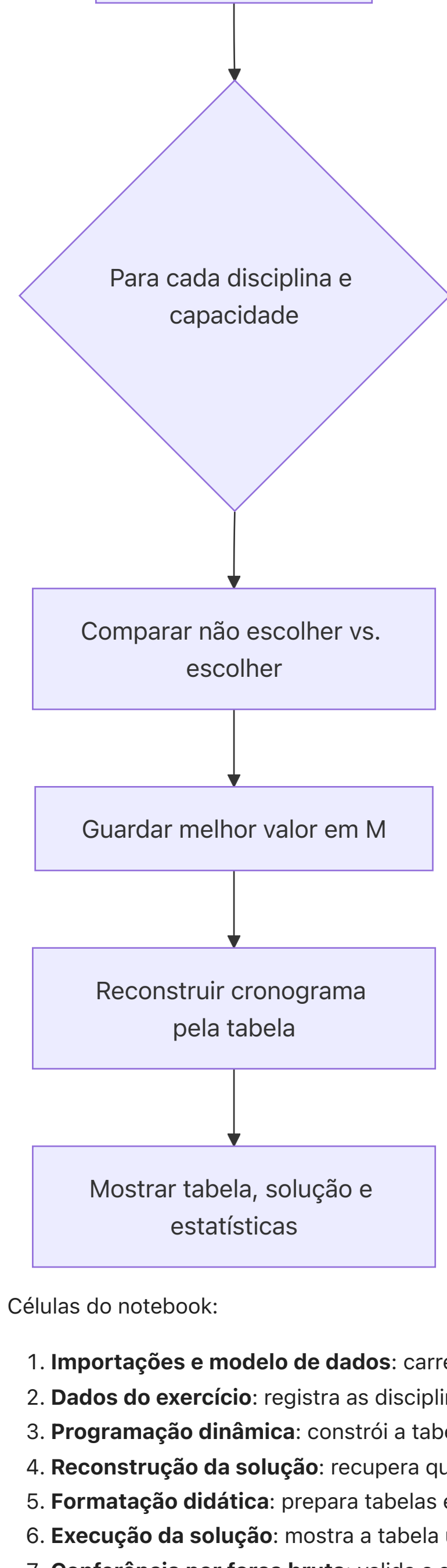
Este notebook resolve o exercício de

Enunciado do exercício: [exercício_1.pdf](#).

A recorrência usada é:

ent que $M[i][k]$ e a m

Montar tabela M



8. Estatísticas e c

- ```
Esta célula importa apenas bibliotecas padrão e define o modelo de dados usado no projeto
from dataclasses import dataclass
from itertools import combinations
from time import perf_counter_ns, process_time_ns
import tracemalloc
from IPython.display import Markdown, display

@dataclass(frozen=True)
class Disciplina:
 """Representa uma disciplina candidata ao cronograma de estudos."""
 nome: str
 tempo: int
 importancia: int
```

```
@dataclass(frozen=True)
class Disciplina:
 """Representa uma disciplina candidata ao cronograma de estudos."""

 nome: str
 tempo: int
 importancia: int

In [2]: # Esta célula registra os dados do enunciado: capacidade de estudo e lista de disciplinas.
 TEMPO_DISPONIVEL = 16

 DISCIPLINAS = [
 Disciplina("ALP", 3, 5),
 Disciplina("PAP", 5, 3),
 Disciplina("CAL", 6, 6),
 Disciplina("BAN", 4, 4),
 Disciplina("REC", 9, 5),
 Disciplina("PPR", 3, 2),
 Disciplina("TEC", 8, 6),
 Disciplina("SOP", 4, 4),
]
```

```
print(f"Quantidade de disciplinas: {len(DISCIPLINAS)}")
```

Tempo disponível: 16 horas  
Quantidade de disciplinas: 8

## Estratégia

```
Esta célula constrói a tabela de programação dinâmica da Mochila Binária (0/1).
```

```

 if disciplina.tempo <= horas:
 com_disciplina = disciplina.importantia + tabela[i - 1][horas - disciplina.tempo]
 tabela[i][horas] = max(sem_disciplina, com_disciplina)
 else:
 tabela[i][horas] = sem_disciplina

 return tabela

In [4]: # Esta célula reconstrói o cronograma ótimo caminhando de trás para frente na tabela.
def reconstruir_cronograma():
 disciplinas = list(Disciplina),
 capacidade = int,
 tabela = list(list(int]),
) -> list(Disciplina):
 """Recupera as disciplinas escolhidas a partir da tabela M já preenchida."""
 escolhidas = []
 horas_restantes = capacidade

 for i in range(len(disciplinas), 0, -1):
 if tabela[i][horas_restantes] != tabela[i - 1][horas_restantes]:
 disciplina = disciplinas[i - 1]
 escolhidas.append(disciplina)

```

```
escolhidas.reverse()
return escolhidas
```

```
def tabela_dg_markdown(disciplinas: list[Disciplina], tabela: list[list[int]] -> str:
 linhas = ["| Disciplina | Tempo (h) | Importância subjetiva (is) |", "|---|---|---|---|"]
 for disciplina in disciplinas:
 linhas.append(f"| {disciplina.nome} | {disciplina.tempo} | {disciplina.importancia} |")
 return "\n".join(linhas)

def tabela_dg_markdown(disciplinas: list[Disciplina], tabela: list[list[int]] -> str:
 cabecalho = ["Item | capacidade"] + [str(horas) for horas in range(len(tabela[0]))]
 alinhamento = ["=" + " | " + "="] * (len(cabecalho) - 1)
 linhas = ["| " + " | ".join(cabecalho) + " |", "| " + " | ".join(alinhamento) + " |"]

 linhas.append(" | 0 disciplinas | " + " | ".join(map(str, tabela[0])) + " |")
 for i, disciplina in enumerate(disciplinas, start=1):
 rotulo = f"i{i} = {disciplina.nome}"
 linhas.append(" | " + rotulo + " | " + " | ".join(map(str, tabela[i])) + " |")

 return "\n".join(linhas)

def cronograma_markdown(escolhidas: list[Disciplina]] -> str:
 linhas = ["| Ordem | Disciplina | Tempo (h) | Importância subjetiva (is) |", "|---|---|---|---|"]
 for ordem, disciplina in enumerate(escolhidas, start=1):
 linhas.append(f"| {ordem} | {disciplina.nome} | {disciplina.tempo} | {disciplina.importancia} |")
```

```
In [6]: # Esta célula executa a solução, mostra a tabela usada e apresenta o melhor cronograma.
tabela = construir_tabela_mochila(DISCIPLINAS, TEMPO_DISPONIVEL)
cronograma_otimo = reconstruir_cronograma(DISCIPLINAS, TEMPO_DISPONIVEL, tabela)

tempo_total = sum(disciplina.tempo for disciplina in cronograma_otimo)
importancia_total = sum(disciplina.importancia for disciplina in cronograma_otimo)

relatorio = f"""
Dados do exercício

{tabela_disciplinas_markdown(DISCIPLINAS)}

Tabela usada na solução

Cada linha representa as primeiras disciplinas consideradas; cada coluna representa uma capacidade de estudo de 0 a ${TEMPO_DISPONIVEL}$ horas.

{tabela_dp_markdown(DISCIPLINAS, tabela)}

Melhor cronograma encontrado

{cronograma_markdown(cronograma_otimo)}
```

```
display(Markdown(relatorio))
```

### Dados do exercício

| Disciplina | Tempo (h) | Importância subjetiva (is) |
|------------|-----------|----------------------------|
| ALP        | 3         | 5                          |
| PAP        | 5         | 3                          |
| CAL        | 6         | 6                          |
| BAN        | 4         | 4                          |
| REC        | 9         | 5                          |
| PPR        | 3         | 2                          |
| TEC        | 8         | 6                          |
| SOP        | 4         | 4                          |

### Tabela usada na solução

Cada linha representa as primeiras disciplinas consideradas

| Item / capacidade | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|-------------------|---|---|---|---|---|---|---|---|---|
| 0 disciplinas     | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 - ALP           | 0 | 0 | 0 | 5 | 5 | 5 | 5 | 5 | 5 |
| 2 - PAP           | 0 | 0 | 5 | 5 | 5 | 5 | 5 | 5 | 8 |
| 3 - CAL           | 0 | 0 | 0 | 5 | 5 | 5 | 6 | 6 | 8 |
| 4 - BAN           | 0 | 0 | 5 | 5 | 5 | 5 | 6 | 9 | 9 |
| 5 - REC           | 0 | 0 | 5 | 5 | 5 | 6 | 9 | 9 | 9 |

7 - TEC                      0 0 0 5 5 5

Melhor cronograma encontrado

| Ordem | Disciplina | Tempo (h) | Importância subjetiva (is) |
|-------|------------|-----------|----------------------------|
| 1     | ALP        | 3         | 5                          |
| 2     | CAL        | 6         | 6                          |
| 3     | BAN        | 4         | 4                          |
| 4     | PPR        | 3         | 2                          |

**Resposta:** estudar ALP, CAL, BAN, PPR, usando 16 horas e obtendo importância 5

```
Esta célula confere a solução por força bruta, testando todas as
def resolver_por_força_bruta(
 disciplinas: list[Disciplina],
 capacidade: int
) -> tuple[int, list[tuple[int, tuple[Disciplina, ...]]]]:
 """Retorna o valor ótimo e todas as combinações ótimas encontradas"""
 melhor_importancia = 0
 combinacoes_otimas = []

 for tamanho in range(len(disciplinas) + 1):
 for combinacao in combinations(disciplinas, tamanho):
```

```

 importancia = sum([disciplina.importancia for disciplina in combinacao])

 if tempo <= capacidade and importancia > melhor_importancia:
 melhor_importancia = importancia
 combinacoes_otimas = [(tempo, combinacao)]
 elif tempo <= capacidade and importancia == melhor_importancia:
 combinacoes_otimas.append((tempo, combinacao))

 return melhor_importancia, combinacoes_otimas

nomes_disciplinas = tuple(nomes_disciplinas)
nomes_cronograma = tuple(nomes_cronograma)
nomes_otimas = tuple(d.nome for d in combinacoes_otimas)

assert importancia_bruta == importancia_total
assert nomes_cronograma in nomes_otimas

print("Conferência por força bruta concluída com sucesso.")
print(f"Importância subjetiva ótima: {importancia_bruta}")

```

```
for tempo, combinacao in combinacoes_otimas:
 nomes = ", ".join(d.nome for d in combinacao)
 print(f"~ {nomes}: {tempo} horas")
```

Importância subjetiva ótima: 17 is  
Combinações ótimas encontradas:  
- ALP, CAL, BAN, PPR: 16 horas  
- ALP, CAL, PPR, SOP: 16 horas

```
Esta célula mede tempo e memória
def medir_execucao(disciplinas: list):
 """Executa a solução uma vez
 tracemalloc.start()
 for disciplina in disciplinas:
 # f. executar
```

```
tabela_medida = construir_tab
reconstruir_cronograma(discip
```

```

fim_parede = perf_counter_ns()
memoria_atual, pico_memoria = tracemalloc.get_traced_memory()
tracemalloc.stop()

return {
 "tempo_parede_ms": (fim_parede - inicio_parede) / 1_000_000,
 "tempo_cpu_ms": (fim_cpu - inicio_cpu) / 1_000_000,
 "memoria_atual_bytes": memoria_atual,
 "pico_memoria_bytes": pico_memoria,
 "celulas_tabela": (len(disciplinas) + 1) * (capacidade + 1),
}

estatisticas = medir_execucao(DISCIPLINAS, TEMPO_DISPONIVEL)
quantidade = len(DISCIPLINAS)
capacidade = TEMPO_DISPONIVEL

estatisticas_markdown = f"""
Estatísticas de execução

Métrica	Valor
Disciplinas (n)	{quantidade}

```

```
Estado da bateria	{estatisticas['estado_bateria']}
Tempo de parede	{estatisticas['tempo_parede_ms']:.4f} ms
Tempo de CPU	{estatisticas['tempo_cpu_ms']:.4f} ms
Memória atual rastreada	{estatisticas['memoria_atual_bytes']} bytes
Pico de memória rastreado	{estatisticas['pico_memoria_bytes']} bytes
```

```
Ordem de complexidade

- **Tempo:** $\Theta(n \cdot C)$, pois cada disciplina é avaliada para cada capacidade inteira de 0 até C .
- **Memória:** $\Theta(n \cdot C)$, pois a solução didática mantém a tabela completa para mostrar a solução e reconstruir o cronograma.
```

```
display(Markdown(estadisticas_markdown))
```

3  
Mittelpunkt

|                                    |            |
|------------------------------------|------------|
| Disciplinas (n)                    | 8          |
| Capacidade (C)                     | 16 horas   |
| Células da tabela $(n + 1)(C + 1)$ | 153        |
| Tempo de parede                    | 0.0447 ms  |
| Tempo de CPU                       | 0.0390 ms  |
| Memória atual rastreada            | 1416 bytes |
| Pico de memória rastreado          | 1456 bytes |